

Architecting Flash-based Solid-State Drive for High-performance I/O Virtualization

Xiang Song, Jian Yang, Haibo Chen

Institute of Parallel and Distributed Systems, School of Software, Shanghai Jiao Tong University

Abstract—Flash-based solid-state drive (SSD) is now being widely deployed in cloud computing platforms due to the potential advantages of better performance and less energy consumption. However, current virtualization architecture lacks support for high-performance I/O virtualization over persistent storage, which results in sub-optimal I/O performance for guest virtual machines (VMs) on SSD. Further, current software-based I/O virtualization violates the “don’t hide power” principle due to inefficient support for some advanced SSD commands (e.g., TRIM) and constrained parallelism, leading to sub-optimal performance and life cycle.

This paper observes that the massive internal parallelism and the block emulation in the flash translation layer (FTL) make flash-based SSD an ideal candidate to support high-performance I/O virtualization for persistent storage. Based on this observation, we propose VFlash, the first storage I/O virtualization architecture that extends existing SSDs with trivial hardware changes to directly expose multiple virtual SSDs to guest VMs. Performance evaluation using a modified FlashSim with two FTL schemes (i.e., DFTL and FAST) shows that VFlash incurs only small performance overhead over native SSDs and can efficiently exploit parallelism.

Index Terms—Solid State Drive, I/O virtualization

1 INTRODUCTION

The attracting features such as low-power consumption and high random access performance of flash-based SSDs make them widely deployed in consumer devices and desktop computers. Recently, due to the continual increase in density and decrease in price of SSDs, major cloud providers also start to integrate flash-based SSDs to their cloud platforms. For example, Amazon has integrated SSDs to virtual machine instances in its elastic cloud computing (EC2) platform.

Unfortunately, I/O virtualization still remains as an obstacle for adoption in performance-sensitive scenarios due to the non-trivial overhead [16] involved in I/O request intervention and device emulation. Such overhead remains large for virtualized SSD in existing virtualization architecture. Even worse, commodity virtual devices exposed to guest VMs violate the “don’t hide power” principle [13]. They usually lack good support for advanced SSD commands like “TRIM” [11] and may serialize parallel accesses from independent VMs.

To illustrate such overhead, we have conducted a simple performance evaluation on different virtual storage options on Xen [5] using an OCZ RevoDrive3 PCIe SSD device. We evaluate the performance of two primary virtual storage options: 1) a file-backed virtual block device (VBD) (an image file is used as a VBD) and 2) a block-level VBD (e.g., /dev/sda1). There are two kinds of VBD drivers: 1) para-virtual driver and 2) emulated driver. We compare them with the performance of physical block devices accessed through the privileged VM (Domain 0). We use three types of workloads generated from iometer [1] using 8 client threads: 90% read, 90% write and 50% read plus 50% write. Figure 1 shows that the para-virtual VBD introduces an average of 1.66X performance overhead due to the cost of device emulation and I/O intervention, while the emulated VBD incurs an average of 4.43X overhead due to the cost of device emulation and I/O intervention, as well as translating virtual block accesses into file image accesses. The

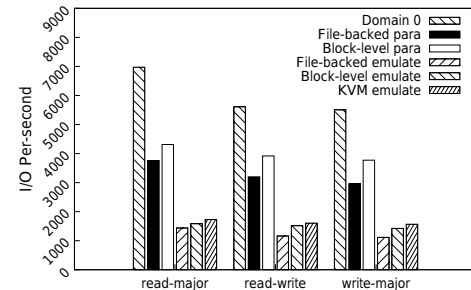


Fig. 1. I/O Performance of native block device, file-backed, and block-level VBD.

latency of a single request is also extended more than 30%.

This paper observes that several features in flash-based SSDs can be retrofitted for I/O virtualization over persistent storage. The massive internal parallelism in channel-level and package-level within SSDs is quite suitable to handle multiple independent accesses from multiple VMs in parallel. Further, current SSDs have already been built with some virtualization features due to the need to emulate a traditional block device interface from a flash interface. We argue that such virtualization features can be easily architected to support high-performance I/O virtualization for storage devices:

- 1) First, one key part in SSDs is address translation in the FTL, which translates logical page address to physical page address. Such a translation essentially matches the address translation in I/O virtualization, making it quite straightforward to reuse such a feature.
- 2) Second, an SSD contains a controller that provides a virtualized interface to above file systems. Further, the NVMe work group has recently announced how to provide an SR-IOV [3] interface on an NVM Express SSD subsystem [2]. Such interface can be used to provide virtual interfaces to guest VMs.
- 3) Third, the massive parallelisms inside an SSD make its internal resources naturally virtualizable by either time-

Corresponding author: Haibo Chen, haibochen@sjtu.edu.cn. This work was supported by China National Natural Science Foundation under grant numbered 61003002 and a grant from Intel.

Date of publication 28 Jul. 2013; date of current version 31 Dec. 2014.

Digital Object Identifier 10.1109/L-CA.2013.22

Authorized licensed use limited to: Jinan University. Downloaded on October 29, 2025 at 06:27:35 UTC from IEEE Xplore. Restrictions apply.

1556-6056 © 2014 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

multiplexing or space partitioning.

- 4) Forth, a virtualized system with multiple VMs simultaneously generates multiple independent streams of accesses makes it more easily to fully exploit the massive parallelism of an SSD.

Based on the above observations, this paper proposes VFlash, the first I/O virtualization architecture for persistent storage based on flash-based SSD. VFlash virtualizes an SSD device and directly provides multiple virtual disks to VMs with very little intervention from the hypervisor. VFlash divides and multiplexes resources of an SSD device and provides each VM with the illusion of a complete SSD by leveraging the internal parallelism, address translation and device emulation features in commodity SSDs. Finally, VFlash does not hide powerful features in SSD, which explicitly exploits the data access parallelism between guest VMs and directly supports advanced commands like TRIM. To support direct assignment of virtual disks in VFlash to guest VMs, VFlash reuses commercially-available hardware support for I/O virtualization called Single Root I/O virtualization (SR-IOV) [3].

To evaluate the performance of VFlash, we modify the Flash-Sim [12] to implement VFlash. We modify two representative FTL schemes (FAST [14] and DFTL [9]) for VFlash and evaluate their performance. Our evaluation results show that VFlash incurs only small performance overhead and can better exploit parallelism over software virtualization.

2 VIRTUALIZING SSD

Based on the characteristics of flash-based SSDs, we propose VFlash, an I/O virtualization architecture that reuses and reprograms key functionalities like address translation and device emulation in SSDs, and leverages existing SR-IOV support for high-performance I/O virtualization.

2.1 VFlash Overview

Generally, there are two key steps for hardware-assisted I/O virtualization: 1) exposing the virtual devices directly to guest VMs and 2) creating multiple virtual SSDs. VFlash leverages the existing SR-IOV protocol to expose multiple virtual functions (VFs) to guest VMs. Figure 2 shows the general architecture of VFlash. Each VF provides necessary interfaces to access a virtual SSD disk without intervention of the hypervisor. The guest VM will access its virtual disk through a dedicated VF interface. VFlash exports a physical function (PF) for the management VM to configure the device as well as the VFs.

To support multiple virtual SSDs, VFlash divides and multiplexes most resources of an SSD device to build the virtual disk control logic, which includes: 1) the pending and retired request queues in the *buffer manager*; 2) the processing resource of the *processor* and 3) the memory resource of the in-SSD *SRAM*. Each time a VF can control only one virtual disk, which exclusively owns one or more flash packages, using its private resources. Specifically, each VF statically owns a private buffer manager with one pending request queue and one retired request queue to buffer requests from/to the virtual disk. Yet the resources of the *processor* and *SRAM* used by an active VF are dynamically divided among the online virtual disks. Each active VF takes a private virtual FTL (vFTL) process to maintain the logical to physical mapping table of its virtual disk, process the requests and do garbage collection.

Each VF can directly accept commands from guest VMs. Hence, each VF can directly handle basic as well as advanced

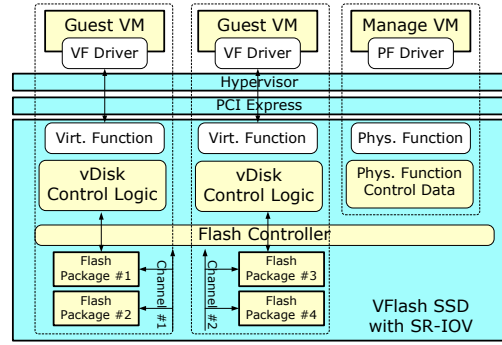


Fig. 2. Overview of VFlash architecture.

SSD commands such as TRIM. This saves the performance overhead from hypervisor intervention. Further, the requests from each VM are served directly by each VF, without the serialization from the management VM. This exposes parallelism in SSDs directly to guest VMs.

2.2 Virtualizing Flash Memory

VFlash currently space-partitions (e.g., at flash package level) the flash memory space of an SSD among virtual disks, which provides better performance isolation and QoS guarantee [15] as well as less hardware complexity.

As the flash memory block footprint of a virtual FTL is limited to a set of flash packages belonging to a specific virtual disk, VFlash can employ existing wear-leveling techniques [4], [6], [10] to even out erases inside a virtual disk. Though there may be possible imbalance of wear-leveling between virtual disks, for our targeted usage scenario of multi-tenant cloud like Amazon EC2, fairness is likely more important as resources are charged per-user based. For example, using B's resource to do garbage collection for A may not be reasonable. Hence, we trade globally balanced wear-leveling for fairness. To ease the wear imbalance problem, we propose two solutions: 1) consolidating VMs with similar disk I/O characteristics on the same machine, which is possible in a large virtualized cloud environment; and 2) migrating a virtual disk from old flash packages to fresh ones, which can be achieved through either offline copying or live VM migration with persistent storage.

2.3 Virtualizing Flash Translation Layer

The virtual disk is accessed and managed by the virtual FTL. When a virtual disk is online, a vFTL is assigned to manage the disk. Each vFTL loads its logical to physical mapping table and data structures in SRAM as its none-virtualized counterpart. The memory resources of SRAM are shared across all active virtual FTLs. Although there are multiple virtual FTLs, the number of physical flash pages is fixed (which decides the maximum number of logical to physical map entries that may exist). Further, the size of the SRAM consumed by any virtual FTL to store its private data is quite small. Thus, VFlash will not further increase the SRAM consumption significantly.

VFlash can be easily integrated with existing FTL schemes. As each vFTL is independent from each other, the FTL scheme used by each vFTL will have no interferences. Thus, changing an FTL scheme for virtualization only needs to wrap the scheme to follow the VFlash resource management specifications and store the mapping table separately from each other

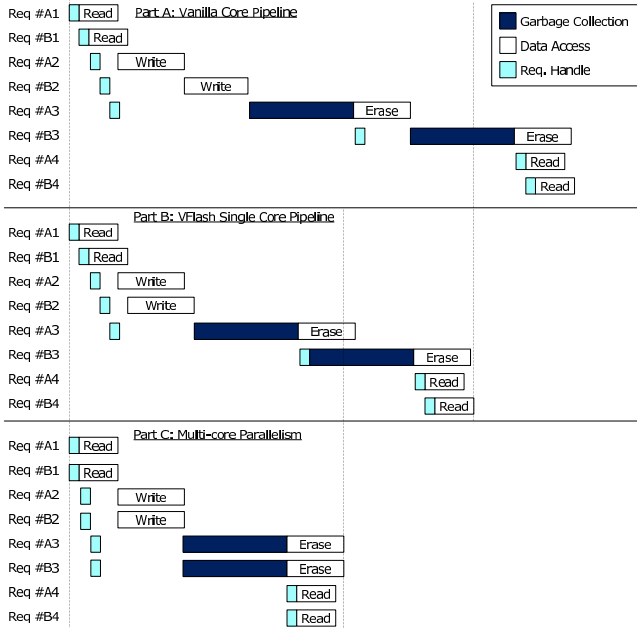


Fig. 3. An example of how requests from guest VM A and B can be parallelized.

when its virtual disk is offline. Here, we take two representative FTL schemes as examples to show how to virtualize state-of-the-art FTLs for VFlash.

Virtualizing Page-level FTL schemes: DFTL Demand-based flash translation layer (DFTL) [9] is a page-level FTL scheme, which maintains the logical to physical mappings in page granularity, which requires lots of SRAM to store them. To reduce SRAM consumption, DFTL selectively caches the most popular mappings in SRAM while stores the mapping table in the data area in flash. To virtualize DFTL, VFlash does not change its mapping management mechanism as well as the garbage collection mechanism. However, in VFlash, a virtual DFTL only allocates pages from the dedicated flash packages for a virtual disk and only uses part of SRAM to cache its in-RAM mappings, which is proportional to its virtual disk size.

Virtualizing Hybrid FTL schemes: FAST Fully associative sector translation (FAST) [14] is a hybrid FTL scheme, which partitions the blocks into data blocks and log blocks. Data blocks are mapped in block granularity and account for most flash memory, while the log blocks are mapped in page granularity and consumes only a small number of blocks. In this case, the mapping table can be stored in SRAM directly. To virtualize FAST, VFlash does not change the mapping and log pages management mechanism and only takes a slight change: A virtual FAST only allocates pages from the dedicated flash packages for a virtual disk and only takes its dedicated flash blocks as log blocks.

2.4 Exploiting Parallelism

State-of-the-art FTLs can explore the parallelism of an SSD by simply pipelining the I/O requests. In the ideal case, all requests can be pipelined. However, in most cases, requests may be serialized when they fall into the same flash die as most existing workloads exploit a sequential stream of data access pattern [7].

On the other side, a virtualized system with multiple VMs

simultaneously generates multiple independent streams of accesses. Suppose there are two guest VMs (VM A and VM B) accessing two virtual disks on an SSD. When the virtual disks are emulated using block-level virtual block devices, requests from both VMs may fall into the same flash die. Thus, independent requests from different VMs are still serialized. Further, complex operations (e.g., garbage collection), which include lots of interactions between the *processor* and the flash controller to manage flash pages, should also be serialized. Part A of Figure 3 shows such an example. Though request 4 from VM A and request 4 from VM B are pipelined as they are accessing different dies, request 2 and 3 from VM A and 2 and 3 from VM B are still serialized when they are accessing the same die.

When the virtual disks are provided through VFlash, requests from both VM A and VM B will never fall into different flash dies. Though the *processor* usually has a single core, the execution of virtual FTLs can still be pipelined. Here, the *processor* uses a round-robin scheduler to execute virtual FTLs. Part B of Figure 3 shows how requests from VM A and VM B can be parallelized through pipeline. Though data accesses from the same VM may be serialized at the flash controller, accesses from different VMs can be handled in parallel as they naturally fall into different flash packages.

The parallelism can further be improved after enhancing the *processor* with multi-core or simultaneous multi-threading (SMT) processors. Part C of Figure 3 shows how requests from VM A and VM B can be processed in parallel on a multi-core or SMT processor. Complex operations (e.g., GC) triggered by different VMs can be processed in parallel. Thus, leveraging a multicore/SMT processor can better utilize the parallelism.

3 EVALUATION

3.1 Simulating VFlash

We extended FlashSim [12], a simulation framework for flash-based storage system, to implement and evaluate the SSD virtualization in VFlash. As the overhead associated with SR-IOV is shown to be very small [8], we mainly focus on evaluating the performance overhead from SSD virtualization. We also omit the device emulation cost for the vanilla runs.

We extended FlashSim with internal parallelism at package-level, where each package can be accessed in constant time and different packages can be accessed simultaneously. Page accesses for a package are queued in RAM if there is pending requests on that package, while operations for different packages will not block each other. Critical operations (e.g., GC) are executed exclusively with other operations, as they will monopolize the FTL process. ALL vanilla SSD results are conducted using the modified FlashSim. VFlash is built upon the modified FlashSim and we have implemented two virtualized FTL schemes on VFlash based on DFTL [9] and FAST [14]. We assume that VFlash is equipped with an SMT processor with 8 threads, thus most concurrent requests from multiple VMs can be distributed in parallel.

3.2 Evaluation Setup

Performance Metrics: The primary parameters for the target SSD are as follows: the total SSD size is 32 GByte with the block size of 128 KByte and the page size of 4 KByte. We assume the SSD has 8 flash packages. Here, we assume a 25 μ s for a page read, 200 μ s for a page write and 1.5 ms for a block erasion.

We omit the cost of querying the in-SRAM mapping table from the performance measurement.

Workloads The Iometer [1] workloads used in analyzing the performance of VFlash are extracted from a Linux-based privileged VM on a Xen VMM running Iometer benchmark including 90% read, 90% write and 50% read using four client threads. The request size used is 4 KByte. We run Iometer on a VM instance with a block-level VBD backed by an OCZ RevoDrive3 PCIe SSD device. The traces are generated in the privileged domain by *blktrace* tool.

3.3 Single Virtual Disk Performance

We provide two versions of VFlash with the virtual DFTL and virtual FAST schemes, respectively. In the evaluation, we assume a 16 GByte virtual disk is allocated from the 32 GByte SSD device and is assigned to an active guest VM through a dedicated virtual function. The block and SRAM resource that the vFTL can use is a half of the total resources in the SSD device. We compare the performance of both versions of VFlash manipulating the virtual disk with the performance of a state-of-the-art SSD controller manipulating a 16 GByte partition using the unmodified DFTL or FAST schemes.

Figure 4 shows the results for both DFTL and FAST. For the DFTL scheme, a virtual DFTL takes 512 KByte RAM as mapping cache, while a vanilla DFTL takes 1 MByte RAM (caches 256K mapping entries). It can be seen that introducing the virtualization support into DFTL adds negligible overhead, through the virtual DFTL suffers a slightly lower map cache hit rate than the vanilla DFTL.

For the FAST scheme, a virtual FAST takes 3% of its virtual disk blocks as the log blocks, while a vanilla FAST takes 3% of disk blocks of the entire SSD as the log blocks. In Figure 4, it can be seen that introducing virtualization support into FAST adds an average performance overhead of 4.56%. This is because virtual FAST has only a half of the random log blocks than vanilla FAST.

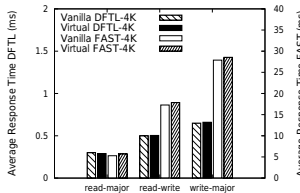


Fig. 4. Performance of virtual and vanilla DFTL and virtual and vanilla FAST under iometer workloads.

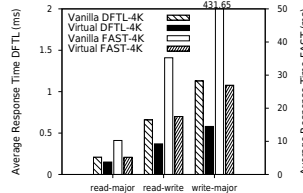


Fig. 5. Performance benefits of exploiting parallelism over software virtualization under iometer

3.4 Benefits of Exploiting Parallelism

We further analyze the potential benefits of exploiting parallelism in VFlash by comparing it with software I/O virtualization. We run two independent Iometer VM instances on two virtual disk devices simultaneously with the same configuration. The two virtual disks locate on the same SSD device. The combined trace collected in the privileged domain is used here. In VFlash, each guest VM directly accesses its virtual disk through a dedicated VF. In a vanilla SSD, virtual disks are emulated through software I/O virtualization technique.

Hence, the requests from two guest VMs are merged at the driver domain.

Figure 5 shows the results. It can be seen in VFlash two VMs can access virtual disks concurrently, which increases the performance by 69.8% (up to 93.3%) for DFTL and 6.67X (up to 16.01X) for FAST on average. The poor performance of FAST is due to the long wait time of each request in the request wait queue. Combining requests from two VMs into one request stream in vanilla FAST aggravates such problem, which result in much higher performance overhead than virtual FAST. Actually, the vanilla FAST with 2-VM using write-major workload failed with a finite request wait queue (10,000 slots by default in FlashSim). Hence, we have to set the queue length to be infinite to complete the test for FAST.

4 CONCLUSION AND FUTURE WORK

In this paper, we made a comprehensive analysis on the characteristics of SSD and I/O virtualization. Our analysis uncovered that many SSDs features resembled the basic functionalities in I/O virtualization, which motivated VFlash. VFlash reused features such as device emulation and address translation as well as massive parallelism in an SSD to create multiple virtual disks and leveraged SR-IOV protocol. Performance evaluation showed that VFlash incurred small performance overhead over devices in native environments.

In our future work, we plan to deploy and evaluate VFlash in a real machine environment using FPGA.

REFERENCES

- [1] "Iometer," <http://www.iometer.org/>.
- [2] "Nvm express," <http://www.nvmexpress.org/>.
- [3] "PCI-SIG Single-Root I/O Virtualization," http://www.pcisig.com/specifications/iov/single_root/.
- [4] N. Agrawal, V. Prabhakaran, T. Wobber, J. Davis, M. Manasse, and R. Panigrahy, "Design tradeoffs for SSD performance," in *Proc. USENIX ATC*, 2008, pp. 57–70.
- [5] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, and A. Warfield, "Xen and the art of virtualization," in *Proc. SOSP*. ACM, 2003, pp. 164–177.
- [6] L. Chang and C. Du, "Design and implementation of an efficient wear-leveling algorithm for solid-state-disk microcontrollers," *ACM Transactions on Design Automation of Electronic Systems (TO-DAES)*, vol. 15, no. 1, p. 6, 2009.
- [7] F. Chen, R. Lee, and X. Zhang, "Essential roles of exploiting internal parallelism of flash memory based solid state drives in high-speed data processing," in *Proc. HPCA*, 2011, pp. 266–277.
- [8] Y. Dong, X. Yang, X. Li, J. Li, K. Tian, and H. Guan, "High performance network virtualization with sr-iov," in *Proc. HPCA*, 2010.
- [9] A. Gupta, Y. Kim, and B. Urgaonkar, "DFTL: a flash translation layer employing demand-based selective caching of page-level address mappings," in *Proc. ASPLOS*, 2009.
- [10] D. Jung, Y. Chae, H. Jo, J. Kim, and J. Lee, "A group-based wear-leveling algorithm for large-capacity flash memory storage systems," in *Proc. CASES*, 2007, pp. 160–164.
- [11] S. Kim, J. Kim, and S. Maeng, "Using solid-state drives (SSDs) for virtual block devices," in *Proc. Workshop on Runtime Environments, Systems, Layering and Virtualized Environments*, 2012.
- [12] Y. Kim, B. Tauras, A. Gupta, and B. Urgaonkar, "Flashsim: a simulator for nand flash-based solid-state drives," in *Proc. of 1st International Conference on Advances in System Simulation*, 2009.
- [13] B. Lampson, "Hints for computer system design," in *Proc. SOSP*. ACM, 1983, pp. 33–48.
- [14] S. Lee, D. Park, T. Chung, D. Lee, S. Park, and H. Song, "A log buffer-based flash translation layer using fully-associative sector translation," *ACM TECS*, vol. 6, no. 3, p. 18, 2007.
- [15] S. Seelam and P. Teller, "Virtual i/o scheduler: a scheduler of schedulers for performance virtualization," in *Proc. VEE*, 2007.
- [16] J. Shafer, "I/O virtualization bottlenecks in cloud computing today," in *Proc. USENIX workshop on I/O virtualization*, 2010.